

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1402

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Total Marks:50

Annexure A

SIMULATION TASK

Code of Conduct:

*I_ Monika.R(19232481)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Monika.R

Annexure A

SIMULATION TASK

QUESTIONS

1. Given the three-address code sequence:

$t1 = x * 1$

$t2 = t1 + y$

$t3 = t2 - 0$

$t4 = t3 * 0$

if $t4 \neq 0$ goto L1

$t5 = y + y$

Simulate the **peephole optimization** process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

2. Partition the following intermediate code into **basic blocks** and construct the corresponding **control flow graph**. Identify all leader statements and mention the rules used to identify them:

(1) $x = a + b$

(2) $y = x * c$

(3) if $y > 50$ goto 6

(4) $z = y - d$

(5) goto 7

(6) $z = y + d$

(7) $w = z * 2$

3. Construct the **DAG** for the following basic block:

$p = a + b$

$q = a + b$

$r = p * q$

$s = a + b$

$t = r + s$

Using the DAG, identify all **common sub-expressions**. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

4. For a target machine with two registers **R0** and **R1**, simulate the working of a **simple code**

generator for the code:

$t1 = a + b$

$t2 = c * d$

$t3 = t1 - t2$

$t4 = e + f$

$t5 = t3 / t4$

Show the **register descriptor** and **address descriptor** after each instruction is processed.

5. For a **RISC-style target machine** with instructions **LOAD, STORE, ADD, SUB, MUL, MOV**, simulate target code generation for the expression:

$(a - b) * (c + d) - e$

Show the complete sequence of target instructions generated. Also explain the **runtime storage management** and **stack frame layout** used during the computation.

Given the three-address code sequence:

$t1 = a + b,$

$t2 = t1 + 0,$

$t3 = t2 * 1,$

if $t3 == 0$ goto L1,

$t4 = t3 + t3$

ANSWERS:

QUESTION 1: Peephole Optimization

Given Three-Address Code

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

Peephole Optimization Process

The given three-address code is optimized by examining a small sequence of instructions and replacing unnecessary operations with simpler equivalent instructions.

Step 1: Algebraic Simplification

The first instruction is:

```
t1 = x * 1
```

Using the algebraic identity:

$$x \times 1 = x$$

Therefore:

```
t1 = x
```

The second instruction becomes:

```
t2 = t1 + y
```

```
t2 = x + y
```

The third instruction is:

```
t3 = t2 - 0
```

Using the identity:

$$x - 0 = x$$

Therefore:

$$t3 = t2$$

Hence:

$$t3 = x + y$$

Step 2: Constant Folding and Simplification

The fourth instruction is:

$$t4 = t3 * 0$$

Using the identity:

$$x \times 0 = 0$$

Therefore:

$$t4 = 0$$

The conditional statement becomes:

if $0 \neq 0$ goto L1

Since $0 \neq 0$ is always false, the condition can never be satisfied. Therefore, the jump to L1 is eliminated.

Step 3: Dead Code Elimination

Since the conditional jump can never occur, the code associated with the unreachable path can be removed if there is no other control flow reaching it.

The instruction:

$$t5 = y + y$$

does not affect the result of the computation leading to the condition or any observable result. Therefore, it is considered dead code and can be eliminated.

Optimized Three-Address Code

$$t1 = x$$

$$t2 = x + y$$

$$t3 = t2$$

$$t4 = 0$$

After further elimination of unnecessary temporary variables:

$t4 = 0$

Since the condition is always false, the conditional jump is also eliminated.

Final Optimized Code

$t = 0$

Optimizations Applied

1. Algebraic simplification: $x \times 1$ was replaced by x .
2. Algebraic simplification: $t2 - 0$ was replaced by $t2$.
3. Constant folding: $t3 \times 0$ was replaced by 0 .
4. Constant condition elimination: $\text{if } 0 \neq 0$ is always false.
5. Dead code elimination: unreachable or unused computations were removed.

Result

The original sequence containing six instructions is reduced to a constant assignment because the expression $t3 \times 0$ always produces zero.

QUESTION 2: Basic Blocks and Control Flow Graph

Given Intermediate Code

1. $x = a + b$
2. $y = x \times c$
3. $\text{if } y > 50 \text{ goto } 6$
4. $z = y - d$
5. $\text{goto } 7$
6. $z = y + d$
7. $w = z \times 2$

Step 1: Identify Leaders

The rules for identifying leaders are:

1. The first statement of the intermediate code is always a leader.
2. Any statement that is the target of a conditional or unconditional jump is a leader.
3. Any statement immediately following a conditional or unconditional jump is a leader.

Applying these rules:

- Statement 1 is a leader because it is the first statement.
- Statement 6 is a leader because it is the target of the conditional jump at statement 3.
- Statement 4 is a leader because it immediately follows the conditional jump at statement 3.
- Statement 7 is a leader because it is the target of the unconditional jump at statement 5.

Therefore, the leaders are:

1, 4, 6, 7

Step 2: Construct Basic Blocks

Basic Block B1

Statements 1–3:

B1:

$x = a + b$

$y = x \times c$

if $y > 50$ goto 6

Basic Block B2

Statements 4–5:

B2:

$z = y - d$

goto 7

Basic Block B3

Statement 6:

B3:

$z = y + d$

Basic Block B4

Statement 7:

B4:

$w = z \times 2$

Step 3: Control Flow Graph

The control flow between the basic blocks is:

$B1 \rightarrow B2$

$B1 \rightarrow B3$

$B2 \rightarrow B4$

$B3 \rightarrow B4$

CFG Representation

START

↓

B1

├── True: $y > 50 \rightarrow B3$

└── False: $y \leq 50 \rightarrow B2$

↓

B4

↑

B3

Therefore, the complete CFG is:

$B1 \rightarrow B2 \rightarrow B4$

$B1 \rightarrow B3 \rightarrow B4$

Step 4: Explanation

The program begins at B1. After calculating x and y, the condition $y > 50$ is evaluated.

- If $y > 50$, control moves directly to B3.
- If $y \leq 50$, control moves to B2.
- B2 executes $z = y - d$ and jumps to B4.

- B3 executes $z = y + d$ and then proceeds to B4.
 - B4 calculates the final value of w .
-

QUESTION 3: DAG Construction and Common Sub-Expression Elimination

Given Basic Block

$p = a + b$

$q = a + b$

$r = p \times q$

$s = a + b$

$t = r + s$

Step 1: Identify Common Sub-Expressions

The expression:

$a + b$

appears three times:

1. $p = a + b$

2. $q = a + b$

3. $s = a + b$

Therefore, $a + b$ is a common sub-expression.

Instead of calculating it three times, it can be calculated once and reused.

Step 2: DAG Construction

The DAG contains the following nodes:

Node N1:

a

Node N2:

b

Node N3:

- (a, b)

The result of N3 is used by:

p

q

s

Then:

Node N4:

$\times (N3, N3)$

This represents:

$r = p \times q$

Finally:

Node N5:

- $(N4, N3)$

This represents:

$t = r + s$

DAG Structure

The important point is that the same node representing $a + b$ is shared by p, q, and s.

Step 3: Optimized Code

Original code:

$p = a + b$

$q = a + b$

$r = p \times q$

$s = a + b$

$t = r + s$

After common sub-expression elimination:

$p = a + b$

$q = p$

$r = p \times p$

$s = p$

$t = r + p$

Since q and s are simply copies of p , they can also be eliminated where possible.

Therefore, the optimized code can be represented as:

$p = a + b$

$r = p \times p$

$t = r + p$

Step 4: Simple Target Code

Assume registers $R0$ and $R1$.

LOAD $R0, a$

ADD $R0, b$

MUL $R0, R0$

ADD $R0, p$

STORE $t, R0$

However, because p is already represented by the value in $R0$, the optimized sequence can be:

LOAD $R0, a$

ADD $R0, b$

MOV $R1, R0$

MUL $R0, R1$

ADD $R0, R1$

STORE $t, R0$

Register Allocation Decisions

1. $R0$ is initially loaded with a .
2. b is added to $R0$, producing $a + b$.
3. The value $a + b$ is copied to $R1$ because it is required again.
4. $R0$ is multiplied by $R1$ to calculate $(a + b) \times (a + b)$.
5. $R1$ is added to $R0$ to calculate the final result.

6. The final result is stored in memory location t.

Thus, the two registers are used efficiently to avoid recomputing the common expression.

QUESTION 4: Register Descriptor and Address Descriptor

Given Code

$t1 = a + b$

$t2 = c \times d$

$t3 = t1 - t2$

$t4 = e + f$

$t5 = t3 / t4$

The target machine has two registers:

R0 and R1

Initial State

Register Descriptor

R0 $\rightarrow$ Empty

R1 $\rightarrow$ Empty

Address Descriptor

a $\rightarrow$ Memory

b $\rightarrow$ Memory

c $\rightarrow$ Memory

d $\rightarrow$ Memory

e $\rightarrow$ Memory

f $\rightarrow$ Memory

Instruction 1: $t1 = a + b$

Generate:

LOAD R0, a

ADD R0, b

Now R0 contains t1.

Register Descriptor

R0 → t1

R1 → Empty

Address Descriptor

t1 → R0, Memory

Instruction 2: $t2 = c \times d$

R1 is free, so use R1.

Generate:

LOAD R1, c

MUL R1, d

Now R1 contains t2.

Register Descriptor

R0 → t1

R1 → t2

Address Descriptor

t1 → R0, Memory

t2 → R1, Memory

Instruction 3: $t3 = t1 - t2$

Both operands are already available in registers.

Generate:

SUB R0, R1

The result is stored in R0.

Therefore:

$R0 = t3$

R1 is no longer required for t2.

Register Descriptor

$R0 \rightarrow t3$

$R1 \rightarrow \text{Empty}$

Address Descriptor

$t1 \rightarrow \text{Memory}$

$t2 \rightarrow \text{Memory}$

$t3 \rightarrow R0, \text{Memory}$

Instruction 4: $t4 = e + f$

R1 is free.

Generate:

LOAD R1, e

ADD R1, f

Now R1 contains t4.

Register Descriptor

$R0 \rightarrow t3$

$R1 \rightarrow t4$

Address Descriptor

$t3 \rightarrow R0, \text{Memory}$

$t4 \rightarrow R1, \text{Memory}$

Instruction 5: $t5 = t3 / t4$

Both operands are already available.

Generate:

DIV R0, R1

The result is stored in R0.

Register Descriptor

R0 → t5

R1 → Empty

Address Descriptor

t3 → Memory

t4 → Memory

t5 → R0, Memory

Final Target Code

LOAD R0, a

ADD R0, b

LOAD R1, c

MUL R1, d

SUB R0, R1

LOAD R1, e

ADD R1, f

DIV R0, R1

STORE t5, R0

QUESTION 5: RISC-Style Target Code Generation

Given Expression

$(a - b) \times (c + d) - e$

Available Instructions

LOAD

STORE

ADD

SUB

MUL

MOV

Step 1: Compute (a – b)

LOAD R0, a

SUB R0, b

Now:

$$R0 = a - b$$

Step 2: Compute (c + d)

LOAD R1, c

ADD R1, d

Now:

$$R1 = c + d$$

Step 3: Multiply the Two Results

MUL R0, R1

Now:

$$R0 = (a - b) \times (c + d)$$

R1 is no longer required.

Step 4: Subtract e

SUB R0, e

Now:

$$R0 = (a - b) \times (c + d) - e$$

Step 5: Store the Final Result

STORE result, R0

Complete Target Instruction Sequence

LOAD R0, a

SUB R0, b

LOAD R1, c

ADD R1, d

MUL R0, R1

SUB R0, e
STORE result, R0

Register Usage

R0 stores the intermediate result:

$$(a - b)$$

Then R0 stores:

$$(a - b) \times (c + d)$$

Finally R0 stores:

$$(a - b) \times (c + d) - e$$

R1 stores:

$$(c + d)$$

Therefore, only two registers are required.

Runtime Storage Management

During execution, the registers are used to store frequently accessed intermediate values.

The storage arrangement is:

$$R0 \rightarrow (a - b) \rightarrow (a - b) \times (c + d) \rightarrow \text{Final Result}$$

$$R1 \rightarrow (c + d)$$

After the multiplication operation, R1 becomes free because its value is no longer required.

The final result is stored in the memory location named result.

Stack Frame Layout

If the expression is evaluated inside a function, the runtime system can use a stack frame.

A simple stack frame can contain:

Stack Frame Component Purpose

Stack Frame Component Purpose

Return Address	Stores the address to return after function execution
Saved Registers	Stores registers that must be preserved
Local Variables	Stores local variables
Temporary Values	Stores intermediate computation results if registers are unavailable
Parameters	Stores function parameters

For this expression, the intermediate values can normally remain in R0 and R1, so no additional temporary stack storage is required.

Given Three-Address Code

```
t1 = a + b
t2 = t1 + 0
t3 = t2 × 1
if t3 == 0 goto L1
t4 = t3 + t3
```

Optimization of the Given Code

Step 1: Algebraic Simplification

```
t2 = t1 + 0
```

Using:

$$x + 0 = x$$

Therefore:

```
t2 = t1
```

Step 2: Algebraic Simplification

```
t3 = t2 × 1
```

Using:

$$x \times 1 = x$$

Therefore:

$$t3 = t2$$

Hence:

$$t3 = t1$$

Step 3: Replace Temporary Variables

Since:

$$t1 = a + b$$

we obtain:

$$t3 = a + b$$

The condition becomes:

if $(a + b) == 0$ goto L1

Step 4: Optimize t4

If the condition is false, the next instruction is:

$$t4 = t3 + t3$$

This can be represented as:

$$t4 = 2 \times t3$$

Therefore:

$$t4 = 2 \times (a + b)$$

Optimized Code

$$t1 = a + b$$

if $t1 == 0$ goto L1

$$t4 = t1 + t1$$

This removes the unnecessary temporary assignments:

$$t2 = t1 + 0$$

$$t3 = t2 \times 1$$

Thus, the optimized code is shorter and more efficient.

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation